MTL458: Operating Systems
Quiz-2 solutions

Duration: 45 minutes **Total: 10 marks**

Question 1: Consider the following program, where the parent thread waits for the child using a semaphore.

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4
5  sem_t s;
6
7  void *child(void *arg) {
8      printf("child\n");
9      sem_post(&s); // signal: child is done
10     return NULL;
11 }
12
13 int main(int argc, char *argv[]) {
14     sem_init(&s, 0, 0); //initialization
15     printf("parent: begin\n");
16     pthread_t c;
17     pthread_create(&c, NULL, child, NULL);
18     sem_wait(&s); // wait for child to finish
19     printf("parent: end\n");
20     return 0;
21 }
```

(a) What happens if we replace line no. 14 (initialization) by `sem_init(&s, 0, 1)`. [1 mark]

(b) Modify the code so that the parent waits for **three child threads** to finish before printing `"parent: end"`. (You are not allowed to use any special library functions other than **semaphores**) [2 marks]

(c) Justify why it works correctly! [2 marks]

Solution. (a) If $X = 1$, the semaphore starts unlocked. The parent's first `sem_wait()` does not block, so it may print "parent: end" before the child finishes — violating the intended synchronization.

(b) Modified code:

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  sem_t s;
5
6  void *child(void *arg) {
7      int id = *(int *)arg;
8      printf("child %d\n", id);
9      sem_post(&s); // signal completion
10     return NULL;
11 }
```

```c

int main(int argc, char *argv[]) {
    pthread_t c[3];
    int id[3];
    sem_init(&s, 0, 0);

    printf("parent: begin\n");

    for (int i = 0; i < 3; i++) {
        id[i] = i + 1;
        pthread_create(&c[i], NULL, child, &id[i]);
    }

    for (int i = 0; i < 3; i++) {
        sem_wait(&s); // wait for each child
    }

    printf("parent: end\n");
    return 0;
}
```

(c) The modified code works because the semaphore is initialized to 0, causing the parent to block until children signal completion.

- Each child calls `sem_post(&s)` once after finishing, incrementing the semaphore count.
- The parent calls `sem_wait(&s)` three times — once for each child — and therefore only proceeds after all three posts have occurred.
- This ensures the parent prints "parent: end" only after all child threads finish execution.

Question 2: Consider the following code.

```c
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *worker1(void *arg) {
    while (1) {
        pthread_mutex_lock(&A);
        // Simulate doing something
        if (pthread_mutex_trylock(&B) == 0) {
            // critical section
            pthread_mutex_unlock(&B);
            pthread_mutex_unlock(&A);
            break;
        } else {
            pthread_mutex_unlock(&A);

        }
    }
}

void *worker2(void *arg) {
    while (1) {
        pthread_mutex_lock(&B);
        if (pthread_mutex_trylock(&A) == 0) {
```

```
            // critical section
            pthread_mutex_unlock(&A);
            pthread_mutex_unlock(&B);
            break;
        } else {
            pthread_mutex_unlock(&B);

        }
    }
}
```

Can this code cause deadlock? If yes, how? Or if not, explain if there is some other problem with the code?

Solution: No, the code cannot deadlock. Both threads acquire locks in opposite order, but since `pthread_mutex_trylock()` is non-blocking, they never wait indefinitely. However, both threads can repeatedly acquire and release locks without entering the critical section — a situation called livelock, where progress halts even though threads remain active. [2 marks]

Question 3: Two threads each call the following function concurrently. Assume the main thread prints the final balance after both complete. The code sometimes prints a negative balance even after using locks.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int balance = 100;

void withdraw(int amt) {
    if (balance >= amt) {
        pthread_mutex_lock(&lock);
        balance -= amt;
        pthread_mutex_unlock(&lock);
    }
}
```

(a) Describe a possible scenario in which the code prints a negative balance. [2 marks]

(b) Correct the code without using any other special library functions. [1 mark]

Solution: (a)

Step	Thread 1	Thread 2	Balance
1	checks `balance >= 80` (true)		100
2		checks `balance >= 50` (true)	100
3	locks and withdraws 80		20
4	unlocks		20
5		locks and withdraws 50	−30
6		unlocks	−30

Thus, both threads pass the check before either updates `balance`, leading to a final negative value.

(b)

```
pthread_mutex_lock(&lock);
if (balance >= amt)
   balance -= amt;
pthread_mutex_unlock(&lock);
```